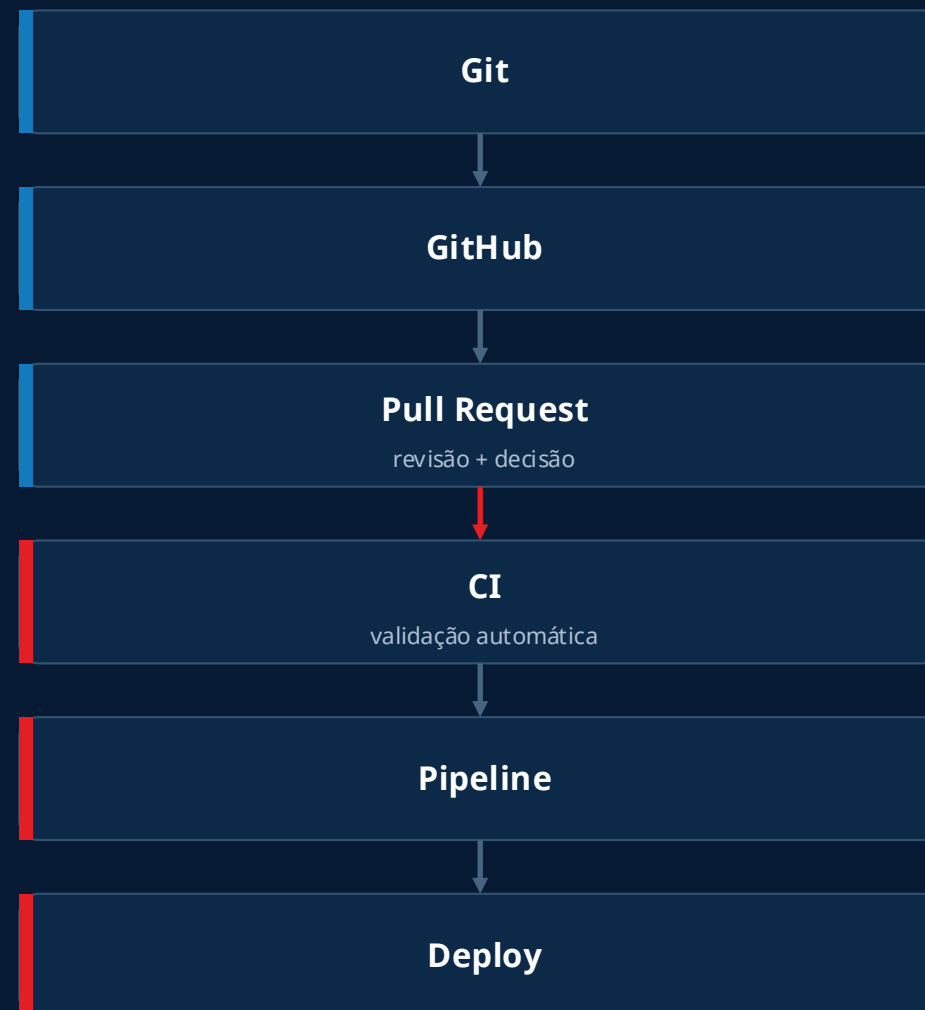


8º PERÍODO

DEVOPS TOOLS

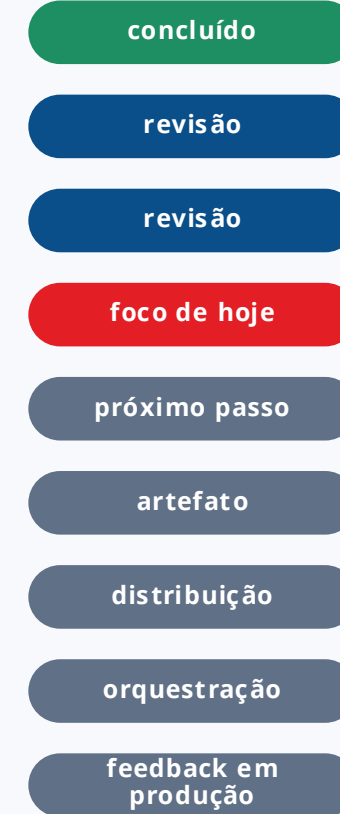
Do trabalho colaborativo
à entrega contínua



PROF. DIOGO RODRIGUES

Onde estamos na disciplina?

Uma trilha: do código ao sistema em operação



Git ≠ GitHub

Tecnologia de versionamento × plataforma de colaboração

GIT

Sistema distribuído de controle de versão

- commits
- branches
- histórico
- merge
- trabalho local

GITHUB

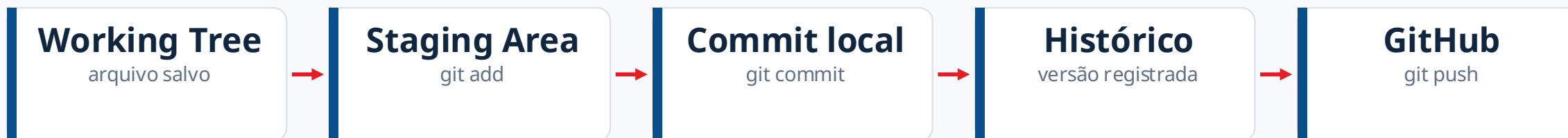
Plataforma baseada em Git

- repositórios remotos
- Pull Requests e revisão
- Issues e governança
- automações

Git é a tecnologia de versionamento. GitHub é uma plataforma que utiliza Git.

O caminho de uma alteração

Do arquivo local ao repositório remoto



ATENÇÃO

**Salvar
arquivo**

≠

**criar
commit**

Commit = unidade lógica e versionada de mudança

O Git essencial

```
$ git status
```

```
$ git diff
```

```
$ git add README.md
```

```
$ git commit -m "docs: adiciona instruções"
```

```
instruções"
```

```
$ git push
```

```
$ git pull
```

```
$ git log --oneline --graph --decorate --all
```

Antes do commit

status + diff
= revisar intenção

Evite automatismo

git add . sem conferir
pode incluir segredos

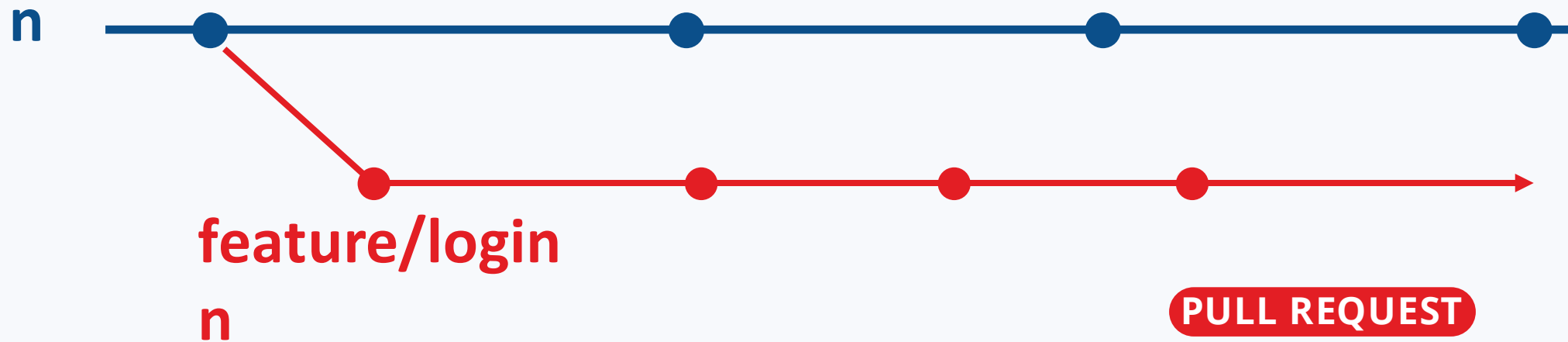
PERGUNTA

O que cada comando faz?

Branches

Linhas independentes de desenvolvimento

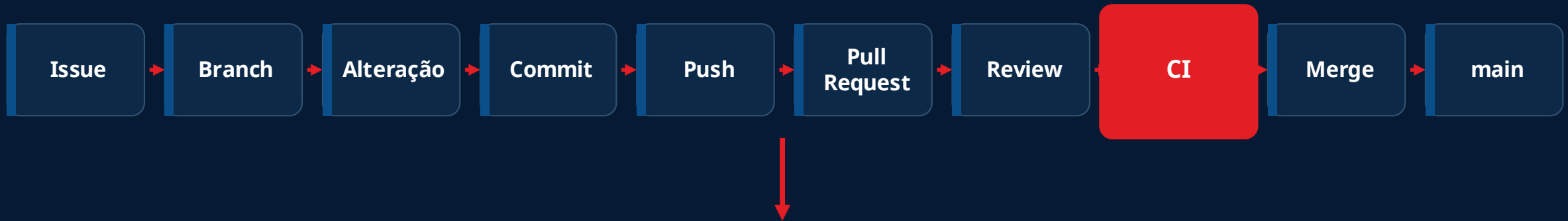
main



Branch cria uma linha independente de desenvolvimento.

Do Issue ao merge

Colaboração com um ponto de verificação automático



O que acontece nesta etapa?

Revisão humana + verificações automáticas

Conflito de merge

Git aponta a divergência; a equipe decide a intenção correta

```
<<<<<<< HEAD
return "versão atual";
=====
return "nova versão";
>>>>>> feature/exemplo
```

Resolver conflito

1. entender as duas intenções
2. escolher/combina conteúdo
3. testar o resultado
4. adicionar e concluir o merge

Não significa simplesmente apagar os marcadores.

Boas práticas rápidas

A revisão termina aqui; a automação começa em seguida

FAÇA

- commits pequenos e coerentes
- mensagens claras
- branches para alterações
- revisar diferenças
- usar .gitignore
- revisar Pull Requests

EVITE

- senhas e .env
- mensagens como “mudanças”
- commits gigantes
- alterações diretas na main
- ignorar conflitos

Da colaboração para a automação

Antes do merge, a máquina pode revisar o que é verificável



LINT

estilo + erros
estáticos

TESTES

comportamento
esperado

BUILD

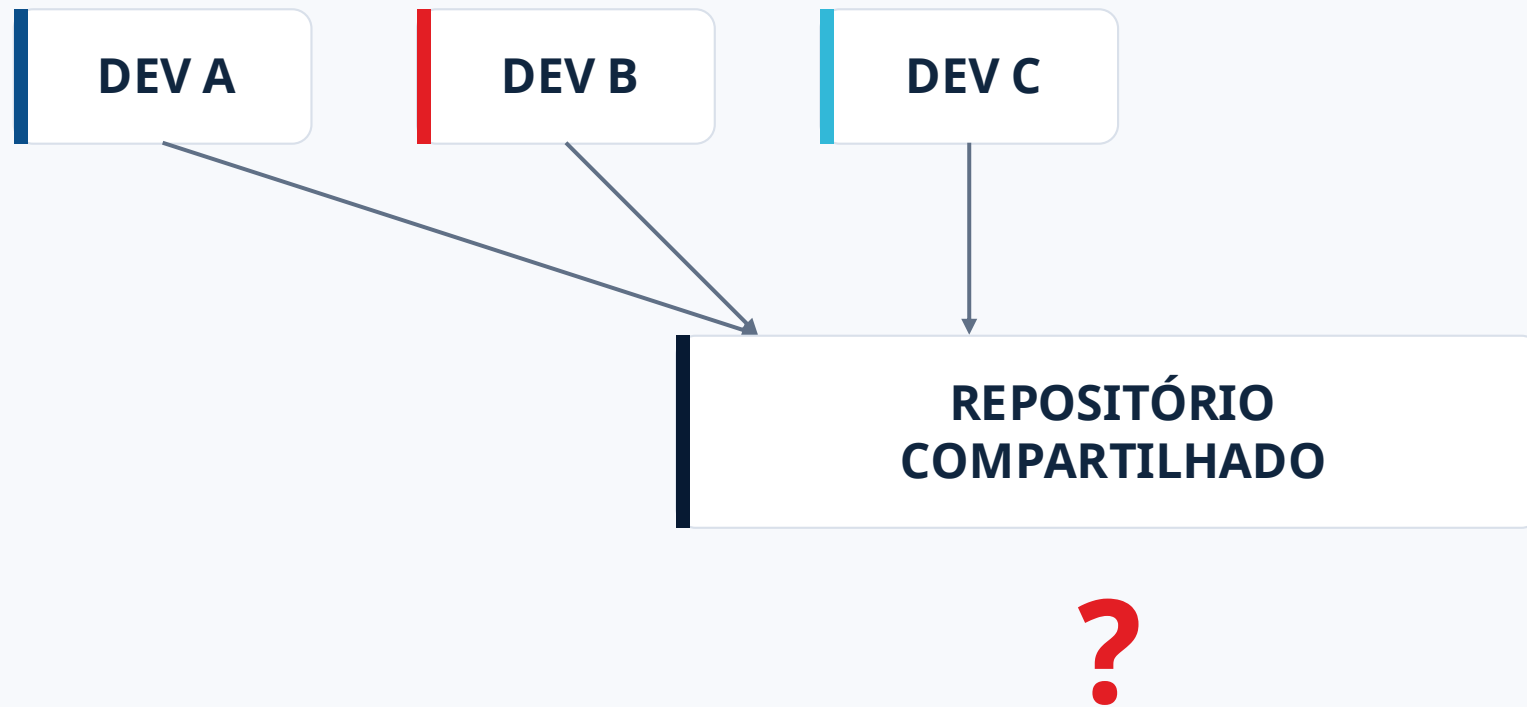
software construível

SEGURANÇA

riscos conhecidos

Que verificações deveriam acontecer automaticamente?

Integrar código não significa que ele funciona



não compila

teste falha

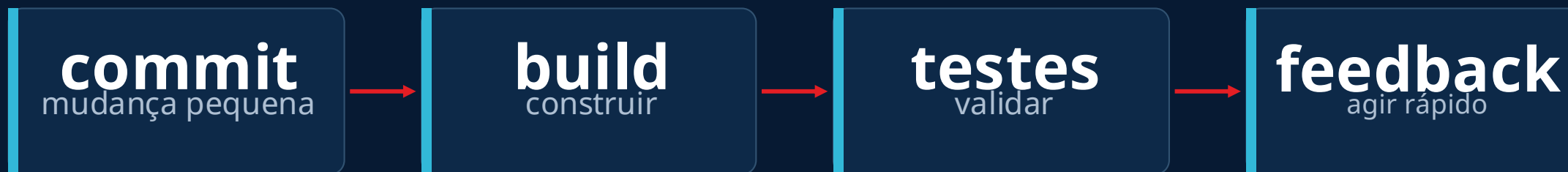
dependência ausente

vulnerabilidade

integração quebrada

CI — Continuous Integration

Integrar pequenas mudanças frequentemente e validá-las automaticamente



CI é uma prática — ferramentas implementam essa prática.

GitHub Actions · Jenkins · GitLab CI · Azure Pipelines

O objetivo do CI

Feedback rápido reduz o espaço de investigação

CENÁRIO A

5 min

CI detecta

erro criado

erro
detectado

CENÁRIO B

3 semanas

produção detecta

erro criado

cliente afetado

Quanto mais cedo detectamos, menor tende a ser o custo de correção.

CI ≠ Delivery ≠ Deployment

Três práticas relacionadas — e diferentes

INTEGRAR

CI

commit
↓
build
↓
testes
↓
feedback

PREPARAR

Continuous Delivery

commit → testes → pacote
pronto para produção
[aprovação] → deploy

IMPLANTAR

Continuous Deployment

commit → testes → pacote
deploy automático
produção

Continuous Delivery

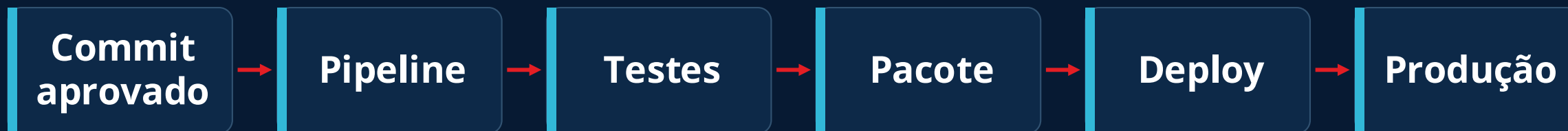
Software sempre pronto para ser implantado



“Contínuo” não significa “sem decisão humana”.

Continuous Deployment

Produção sem aprovação manual



TUDO AUTOMATICAMENTE

Não é “melhor” por definição: risco · maturidade · negócio · compliance · criticidade

Não existe uma pipeline universal

A arquitetura do fluxo acompanha o risco

MENOR RISCO

Site simples

Lint

↓

Teste

↓

Build

↓

Deploy

MAIOR RISCO

Sistema crítico

Lint → unitários → build

↓

Security scan → integração

↓

Homologação → aprovação

↓

Deploy controlado

As peças de uma pipeline

Trigger

inicia execução

Stage / Job

agrupa trabalho

Step

comando individual

Runner / Agent

máquina que executa

Artifact

saída produzida

Gate

condição para prosseguir

Environment

destino da implantação

evento → trabalho → evidência → decisão → destino

O que inicia uma pipeline?

Pipelines são normalmente orientadas a eventos

push

código enviado

Pull Request

validação pré-merge

tag

versão marcada

release

publicação

schedule

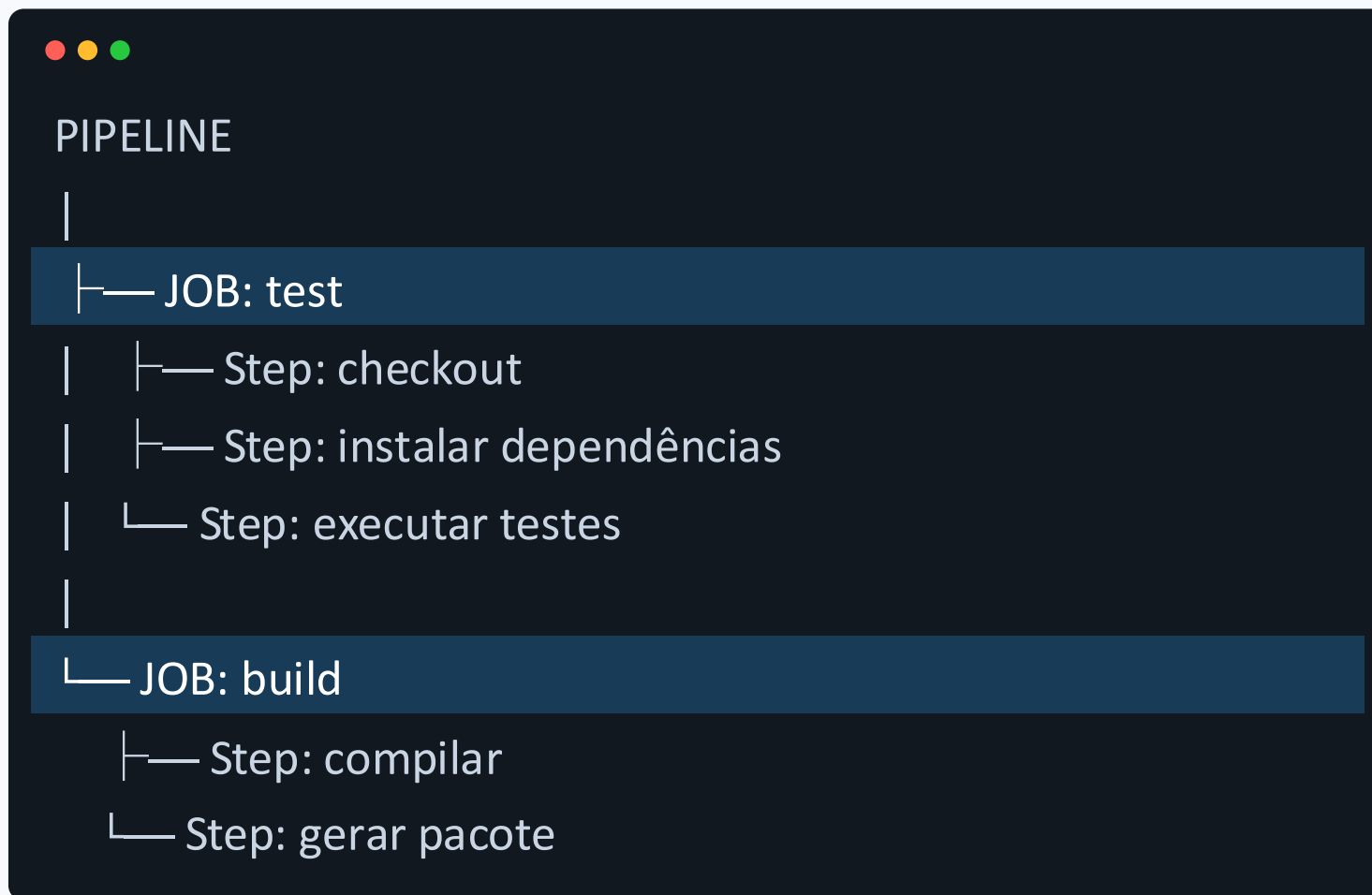
horário recorrente

manual

decisão explícita

Jobs, Steps e Runners

Workflow → Jobs → Steps



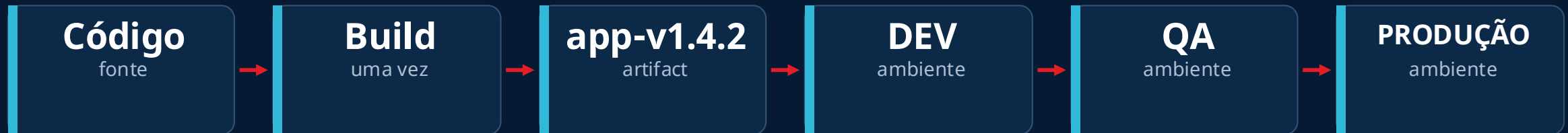
RUNNER

Máquina que executa os steps de um job.

Dependência

build pode esperar test — ou rodar em paralelo quando não depender dele.

Build uma vez. Promova o mesmo artefato.



MESMO ARTEFATO

Recompilar por ambiente cria diferenças difíceis de rastrear.

Não use apenas latest

Qual versão está em produção?

SEM RASTRO

RUIM

meuapp:latest

ponteiro mutável
origem ambígua

IDENTIFICÁVEL

MELHOR

meuapp:1.4.2

ou

meuapp:a8f92c1

Versão explícita favorece rastreabilidade e rollback.

Gates

Podemos impedir que a pipeline avance



teste aprovado

cobertura mínima

análise de segurança

aprovação humana

janela de implantação

Fail fast

Falhe cedo e barato

PIPELINE RUIM



PIPELINE MELHOR



Rápido + barato primeiro

Pipeline recomendada

Exemplo — não receita universal

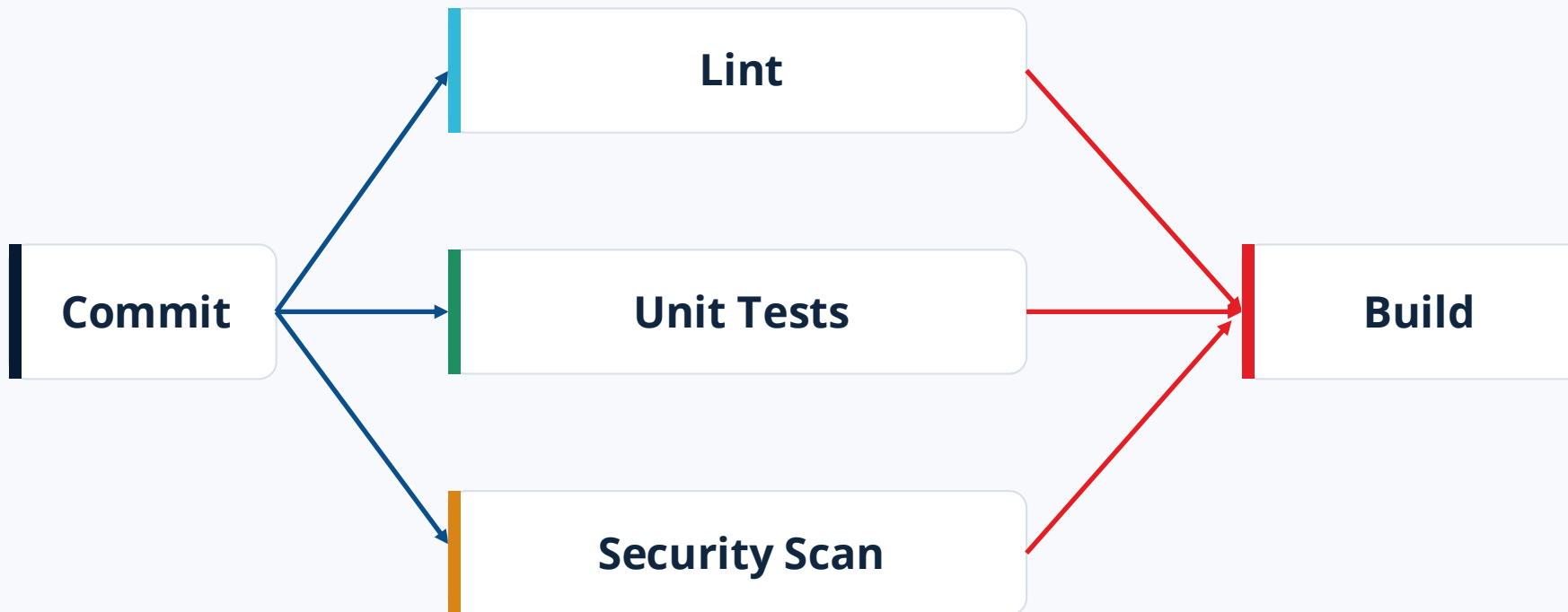


Onde podemos paralelizar?

dependências definem a resposta

Paralelismo

Projetar para reduzir o tempo de feedback



Trade-off

menos tempo
×
mais runners
×
mais custo

Pipeline verde significa software perfeito?

NÃO.

PIPELINE VERDE ✓

testes ausentes

requisito errado

cenário não previsto

problema de infraestrutura

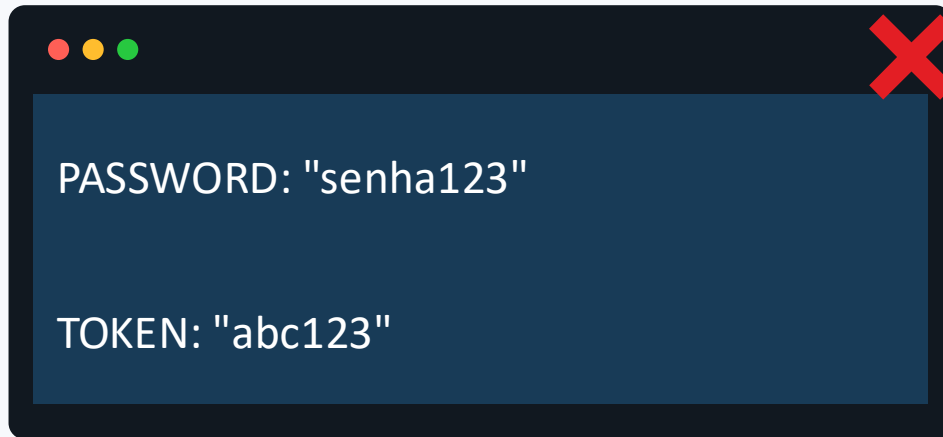
falha não detectada

problema em produção

Automação reduz risco. Não elimina risco.

E os segredos?

Credenciais não pertencem ao código



Nunca commitar .env, tokens ou chaves.

Do CI ao deploy

Passamos nos testes. E agora?



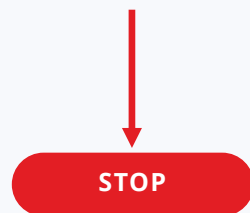
Cada seta pode conter condições, políticas e riscos.

Agora: estratégias para reduzir impacto

Estratégia: Recreate

Parar a versão atual; iniciar a nova

V1



V2



Características

- simples
- fácil de entender
- pode causar indisponibilidade

Rolling Update

Substituição gradual



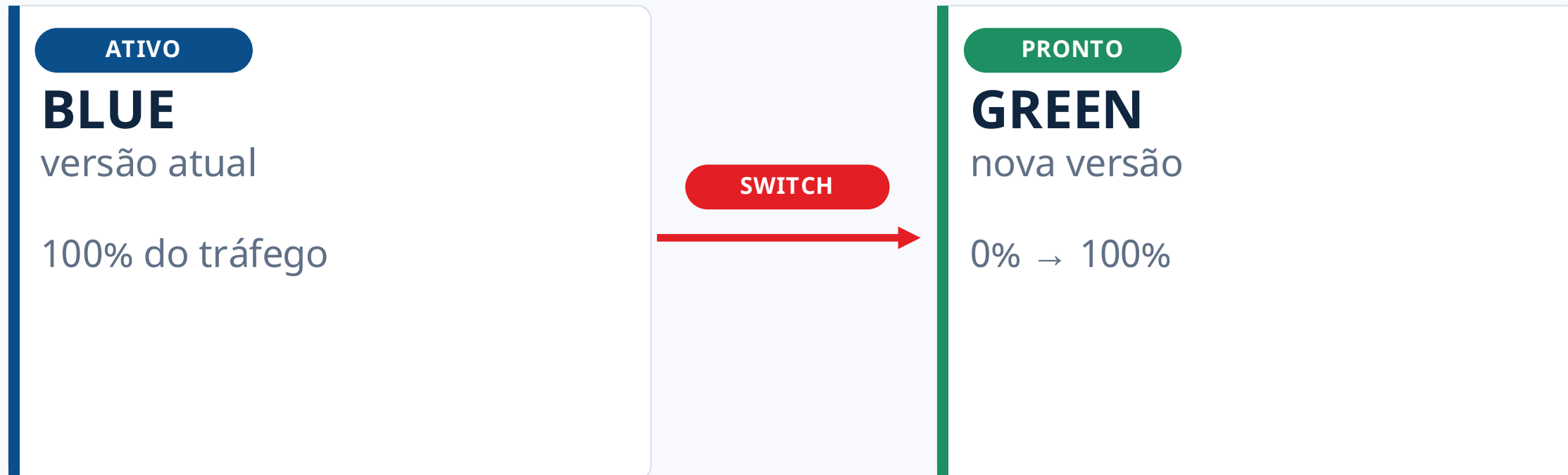
Características

- reduz indisponibilidade
- versões coexistem por um período
- exige compatibilidade temporária

V1 e V2 conseguem conversar?

Blue / Green

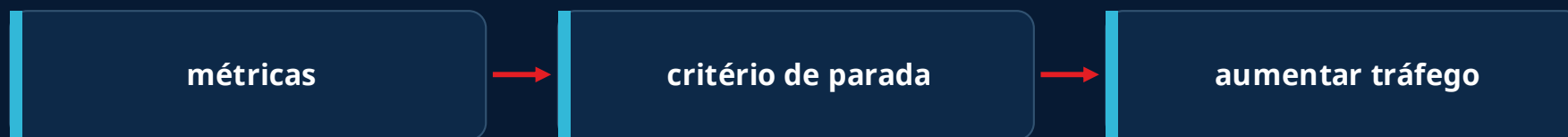
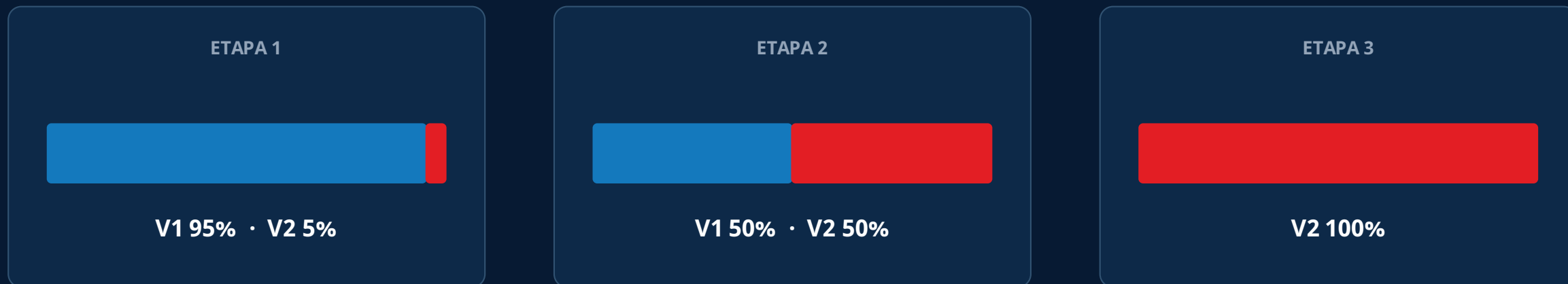
Dois ambientes; uma troca de tráfego



rollback rápido · dois ambientes · maior custo

Canary

Exposição gradual + decisão por métricas



Comparando estratégias

A escolha depende de contexto

Estratégia	Downtime	Complexidade	Rollback	Infraestrutura
Recreate	possível	baixa	médio	baixa
Rolling	baixo	média	médio	média
Blue/Green	muito baixo	alta	rápido	alta
Canary	muito baixo	alta	controlado	alta

Disponibilidade · custo · compatibilidade · capacidade operacional

Rollback

Automatizar deploy exige saber voltar

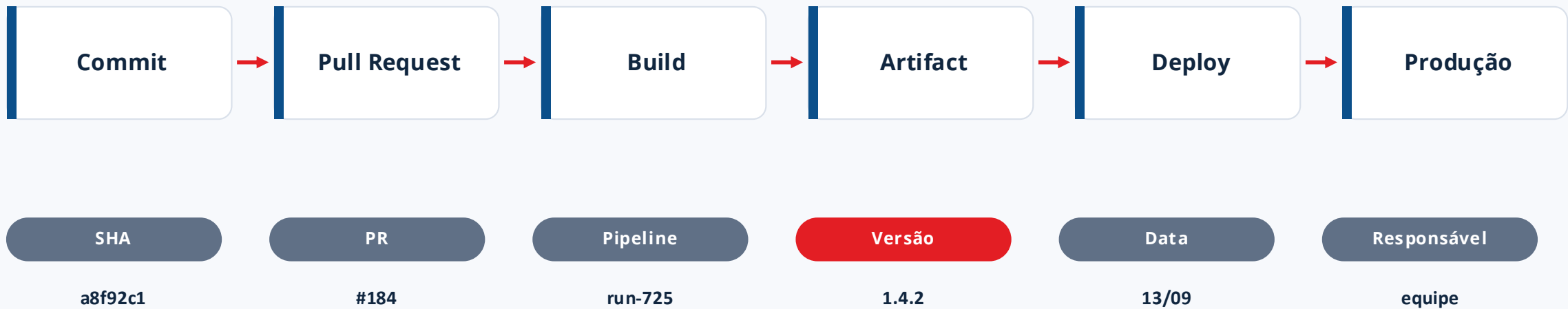


voltar rápido

Aplicação pode voltar. Dados exigem compatibilidade e plano próprio.

Rastreabilidade

Quem colocou o quê em produção?



Uma cadeia de evidências para operação, auditoria e incidentes.

Erros comuns em CI/CD

**pipeline extremamente
lenta**

testes instáveis

tarefas manuais

somente tag latest

segredos no YAML

artefato reconstruído

ausência de rollback

sem responsável

Qual deles faria a equipe ignorar a pipeline?

A pipeline também é software

Pipeline como produto, não como arquivo esquecido

versionamento

histórico

manutenção

evolução

testes

confiança

monitoramento

sinais

documentação

uso

responsáveis

ownership

Evolui com o sistema, os riscos e a organização.

Atividade em grupo

Projete uma pipeline e defenda suas decisões

10-15 MIN

lint

teste unitário

build

security scan

teste integração

publicação

aprovação

deploy

smoke test

ENTREGA DO GRUPO

1. ordenar etapas
2. justificar a ordem
3. identificar paralelismo
4. definir fail fast
5. posicionar aprovação

Não existe uma única resposta correta.

Cenário da atividade

Sistema de comércio eletrônico em produção 24x7

WEB

interface pública

APIs

serviços

PAGAMENTOS

alto impacto

24x7

sem janela fácil

AUTOMAÇÃO

testes existentes

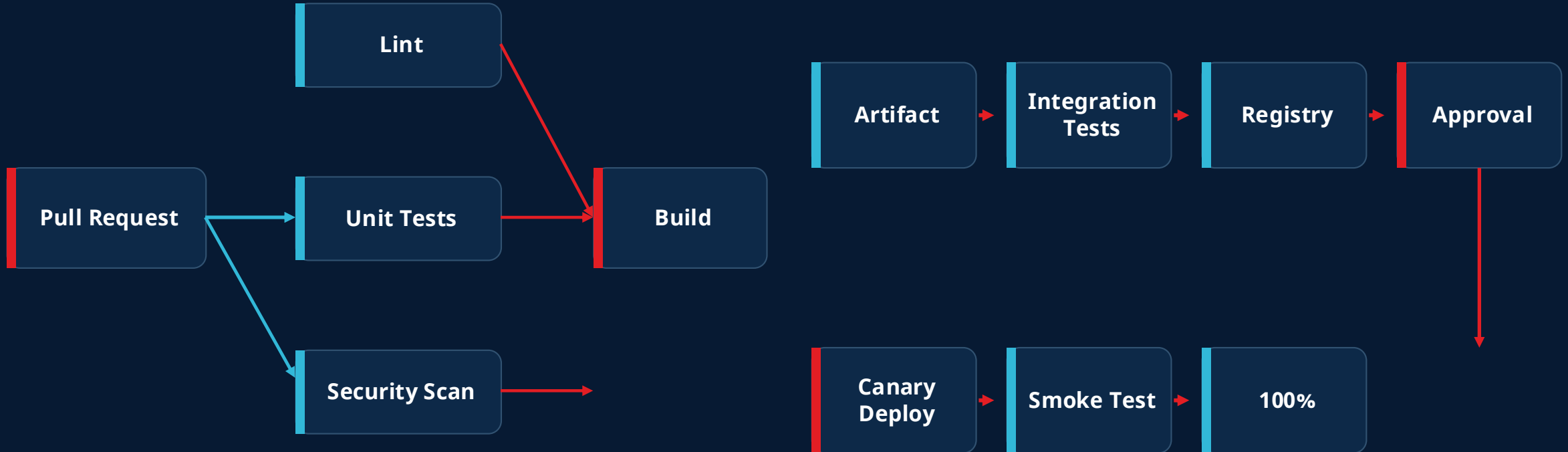
DADOS

sensíveis

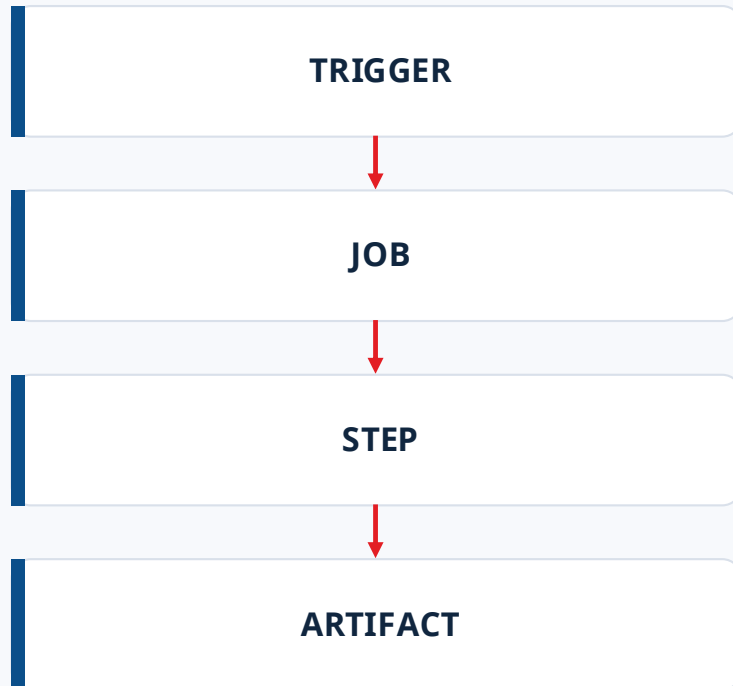
Como vocês projetariam a pipeline?

Uma possível solução

Compare decisões — não procure uma receita



De pipeline conceitual para GitHub Actions



conceito



configuração executável

Você consegue explicar?

Responda antes de avançar

01

CI e Deployment são iguais?

02

Delivery exige deploy automático?

03

O que é um artifact?

04

O que é um gate?

05

Por que falhar cedo?

06

Pipeline verde garante perfeição?

07

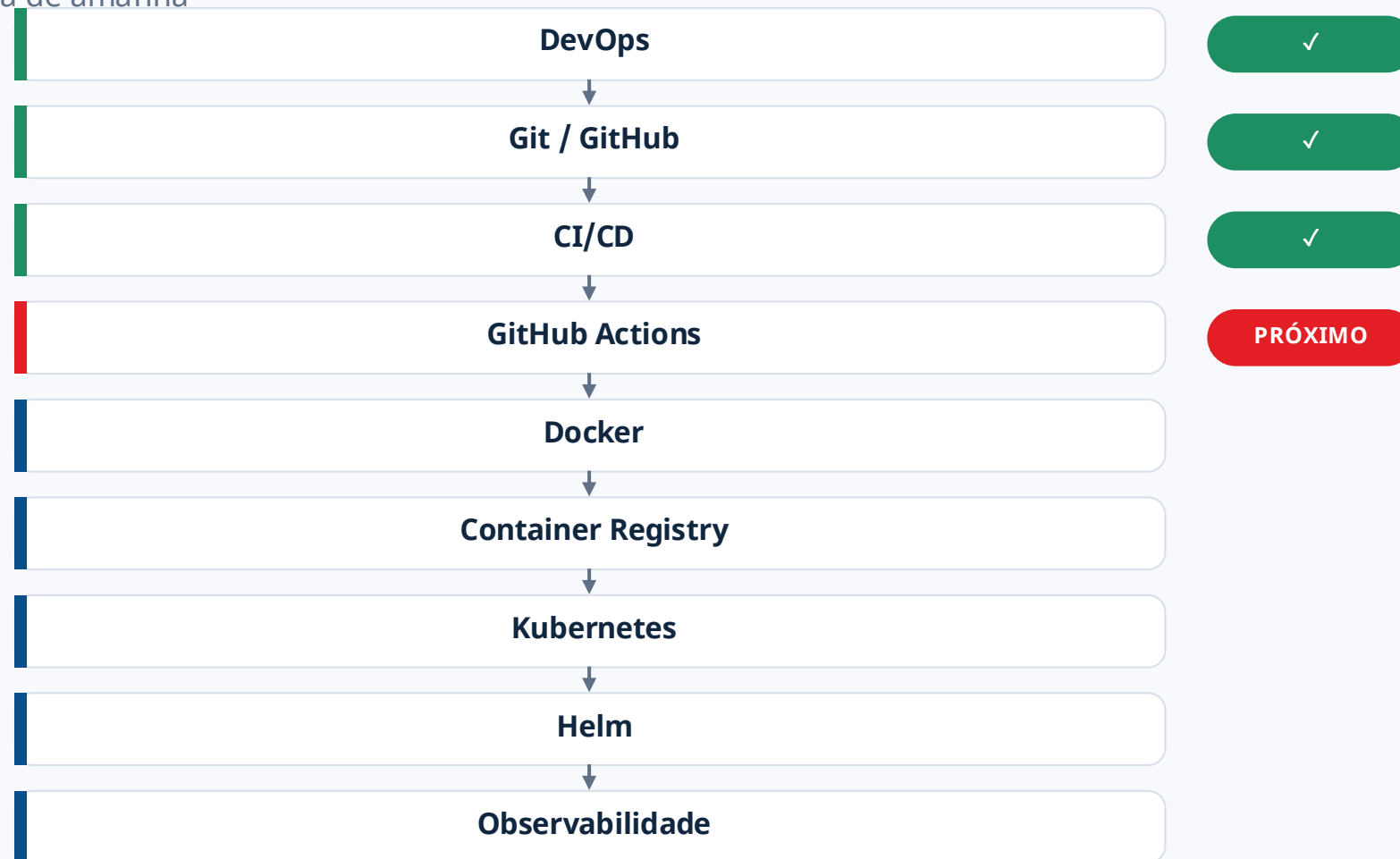
Vantagem de Blue/Green?

08

Quando Canary interessa?

Mapa da disciplina

Da prática de hoje à plataforma de amanhã



7 ideias essenciais

01

Git organiza mudanças; GitHub organiza colaboração.

02

Pull Requests criam um ponto de revisão.

03

CI busca integração e feedback frequentes.

04

Delivery mantém o software pronto para produção.

05

Deployment automatiza a ida até produção.

06

Pipeline é fluxo projetado, não receita universal.

07

Automação precisa de segurança, rastreabilidade e recuperação.

GITHUB ACTIONS

Vamos transformar o desenho em código

Hoje projetamos pipelines.
Na próxima etapa, vamos executá-las.



commit → feedback → confiança → entrega